Solution Intent Guidelines

1. Purpose

A Solution Intent is the reviewable contract between the problem and the implementation.

It explains **what will be built, why this design is appropriate, what assumptions it depends on, how success will be tested, and how the system will be operated safely**.

It is not a marketing proposal and it is not a low-level implementation manual.

2. When to create it

Create the first version after SCQ/discovery and before significant implementation. Update it when major assumptions or architecture decisions change.

3. Required structure

3.1 Document control

- Project name
- Customer / stakeholder
- Prepared by
- Reviewers
- Version
- Date
- Status: Draft / In Review / Approved / Superseded

3.2 Executive summary

In one page or less:

- current problem;
- desired outcome;
- proposed solution;
- key value;
- largest constraints/risks;
- requested decision.

3.3 Situation and challenge

Summarize approved SCQ findings. Do not reintroduce unsupported assumptions.

3.4 Goals and non-goals

Use measurable goals.

Example:

- Process a normal ETL batch in <=30 minutes.
- Make same-day reruns idempotent.
- Provide failure alerting and an operator runbook.

Non-goal example:

- The first release will not provide a graphical ETL workflow editor.

3.5 Scope

Separate:

- in scope;
- out of scope;
- future scope.

3.6 Requirements summary

Reference requirement IDs rather than rewriting them inconsistently.

ID	Requirement	Priority	Acceptance method
BR-001	…	Must	…

3.7 Current-state architecture

Describe existing systems, dependencies, data flow and pain points.

3.8 Proposed target architecture

Provide a diagram and explain each component.

For cloud solutions, review architecture against relevant Well-Architected concerns:

- operational excellence;
- security;
- reliability;
- performance efficiency;
- cost optimization;
- sustainability.

3.9 Data flow and data model

Document:

- input sources;
- transformations;
- storage;
- retention;
- identifiers/idempotency rules;
- sensitive-data handling;
- schema evolution.

3.10 Interfaces and API contracts

For services/APIs include:

- endpoints/events;
- request/response formats;
- authentication/authorization;
- error model;
- pagination;
- rate limits;
- versioning;
- retry/idempotency semantics.

3.11 Security design

At minimum:

- identity and least privilege;
- secrets management;
- encryption in transit/at rest;
- network exposure;
- dependency/image scanning;
- input validation;
- audit logging;
- data classification and retention;
- threat scenarios relevant to the project.

3.12 Reliability and recovery

Define:

- expected availability;
- failure modes;
- retries/timeouts;
- idempotency;
- backup/restore where applicable;
- RTO/RPO if applicable;
- rollback strategy;
- degradation behavior.

3.13 Observability

Define the minimum telemetry needed to operate the solution:

- health checks;
- metrics;
- logs;
- traces where useful;
- dashboards;
- alert rules;
- alert ownership;
- runbook links.

3.14 CI/CD and release strategy

Document:

- source-control workflow;
- build and test stages;
- security checks;
- artifact/version strategy;
- environment promotion;
- secrets handling;
- deployment mechanism;
- rollback.

3.15 Infrastructure and configuration

State:

- what is managed by IaC;
- environment differences;
- configuration strategy;
- state/storage approach;
- backup of critical configuration;
- drift-management method.

3.16 Testing strategy

Include:

- unit tests;
- integration tests;
- contract/API tests;
- end-to-end tests;
- negative/error-path tests;
- performance tests when required;
- security tests;
- restore/recovery tests;

- UAT.

3.17 Migration / rollout

For replacement systems explain:

- initial data/config migration;
- parallel-run period if needed;
- cutover steps;
- validation;
- rollback triggers;
- rollback procedure.

3.18 Assumptions

Every material assumption should have:

ID	Assumption	Impact if false	Owner	Validation date

3.19 Dependencies

Record people, teams, accounts, APIs, datasets, network access, licenses and approvals required.

3.20 Risks

Use probability/impact, owner and mitigation.

3.21 Open questions

Track questions to closure. Unresolved questions that can invalidate the design must block approval.

3.22 Alternatives considered

For major choices, show at least one credible alternative and why it was not selected.

Example:

Option	Advantages	Disadvantages	Decision

3.23 Effort estimate and milestone plan

Avoid false precision. Break the estimate into work packages and identify assumptions.

Example:

Work package	Estimate	Dependencies	Exit condition
Discovery/design	2-3 days	Stakeholder availability	Design approved
MVP vertical slice	3-5 days	Accounts/data	Core flow works

3.24 Acceptance criteria

The final section must link back to requirements and define evidence.

3.25 Approval

Record reviewer decision and conditions.

4. Diagrams

Use Mermaid, diagrams.net, PlantUML or another versionable format where practical. A useful diagram shows boundaries, trust zones, major data paths and dependencies rather than decorative icons.

5. Quality criteria

A Solution Intent is ready for approval when:

- business outcome is measurable;
- requirements are traceable;
- architecture is understandable without verbal explanation;
- major trade-offs are explicit;
- security/reliability/operations are designed, not postponed;
- open questions are either closed or accepted as risks;
- acceptance criteria can be executed;
- estimate includes dependencies and assumptions;
- rollback/recovery is defined where failure matters.

6. Template

Use `../templates/SOLUTION_INTENT_TEMPLATE.md`.